Natural Language Processing - Lecture 03

Lecture overview

Lecture 03 introduces language modeling as the task of predicting the next word in a sequence or assigning probabilities to linguistic sequences, with applications ranging from spell checking and speech recognition to text generation. The lecture details n-gram models, which estimate probabilities by counting occurrences in a corpus using the Chain Rule and the Markov Assumption to limit context to the last $N-1$ words. While Maximum Likelihood Estimation (MLE) calculates these probabilities based on raw counts, the approach suffers from data sparsity, where many theoretically possible n-grams have zero counts in training data. This leads to the "zero probability problem," rendering evaluation metrics like perplexity impossible for unseen sequences and necessitating the use of log probabilities to prevent numerical underflow during calculation.

To address data sparsity, the lecture focuses on smoothing techniques that redistribute probability mass from frequent events to rare or unseen ones, summarized as "stealing from the rich to give to the poor." While Add-One (Laplace) smoothing ensures non-zero probabilities by adding one to all counts, it is often considered a blunt instrument that distorts distributions too severely for n-gram models. Consequently, more sophisticated methods like Backoff and Interpolation are preferred; Backoff recursively falls back to lower-order n-grams when higher-order evidence is insufficient, while Interpolation linearly combines probabilities from unigram, bigram, and trigram models using context-conditioned weights. The lecture emphasizes that proper smoothing must ensure probabilistic soundness, such as through Katz backoff, to maintain a valid probability distribution where the sum of probabilities equals one.

Slide 1: Natural Language Processing:

This lecture introduces the topic of Natural Language Processing, specifically focusing on language modelling. The course is identified as H02B1A, taught by Miryam de Lhoneux at the Department of Computer Science, KU Leuven. The presentation slides are adapted from Jurafsky and Martin (2025).

Slide 2: Introduction

This introduction outlines the course structure, covering N-grams, generation via sampling, evaluation, smoothing, and a summary. The session focuses on Natural Language Processing (NLP) with an emphasis on language modeling as the foundational topic.

Slide 3: Introduction

Language modelling in Natural Language Processing (NLP) involves predicting the next word in a sequence. This concept is illustrated by the example sentence "Unicorns bring joy and wonder to everyone they...", where the model must select the most probable continuation from options such as "meet," "know," "see," or "the day."

Slide 4: Introduction

Language models are fundamental tools in natural language processing that assign probabilities to linguistic sequences. Specifically, they can calculate the likelihood of any potential next word within a context or determine the probability of an entire sentence. This capability forms the basis for understanding and generating human language computationally.

Slide 5: Introduction

Language modelling evaluates word sequence probability to enable grammar and spell checking, such as distinguishing "improve" from "improove." It also refines speech recognition by selecting probable sequences like "I will be back soonish" over acoustically similar but unlikely alternatives. Fundamentally, large language models (LLMs) operate by predicting words sequentially to generate text.

Slide 6: N-grams

Slide 6 presents the outline for the N-grams section within the context of Natural Language Processing and language modeling. The agenda covers five key topics: an introduction to N-grams, generation via sampling, evaluation methods, smoothing techniques, and a final summary. This slide serves as a structural overview without providing detailed definitions or examples for these concepts.

Slide 7: N-grams

N-gram language models focus on two prediction goals: calculating the probability of an entire sentence sequence to assess grammaticality, or predicting the probability of an upcoming word given a preceding sequence. These probabilities are estimated directly from a corpus by counting and dividing occurrences, rather than using neural methods. The model determines the likelihood of a specific next word based on the history of previous words within that text collection.

Slide 8: N-grams

Estimating sentence probabilities by simply counting and dividing is unreliable because corpora lack sufficient data for the vast number of possible sentences. Consequently, language modeling relies on the chain rule of probability to decompose full sequence estimation into manageable conditional probabilities. This approach addresses the sparsity issue where rare or unseen word sequences would otherwise result in zero probability estimates.

Slide 9: N-grams

The chain rule addresses data sparsity in language modeling by expressing the joint probability of a word sequence as the product of each event's conditional probability given its preceding events. This generalization models a sequence where the probability of any event depends on all prior events, ranging from x_1 for x_2 up to x_{n-1} for x_n. Consequently, the probability of an entire sentence is calculated as the product of the probabilities of each individual word conditioned on the preceding context.

Slide 10: N-grams

The chain rule decomposes joint probabilities into conditional probabilities, expressed as P(A, B) = P(A)P(B|A). This generalizes to n variables where P(x1, ..., xn) equals the product of P(x1) and subsequent terms P(xi|x1, ..., xi-1). In Natural Language Processing, this mathematical foundation supports language modeling by calculating the probability of word sequences.

Slide 11: N-grams

N-grams apply the chain rule to calculate the joint probability of words in a sentence, expressed as $P(w_{1:n}) = \prod_{k=1}^{n} P(w_k | w_{1:k-1})$. This decomposes the probability into a sequence of conditional probabilities where each word depends on all preceding words. For example, the sentence "Unicorns bring joy and wonder" is calculated by multiplying the probability of "Unicorns" by the conditional probabilities of subsequent words given their specific preceding context.

Slide 12: N-grams

To address data sparsity in language modeling, the Markov Assumption restricts word prediction to a limited context, such as the last two words for 3-grams. Instead of conditioning on the entire preceding sequence, the probability of "and" depends only on "bring joy," and "wonder" depends only on "joy and." This simplification reduces the computational burden while approximating the full sentence probability as a product of these localized conditional probabilities.

Slide 13: N-grams

N-gram language models apply the Markov assumption to approximate word probabilities by restricting context to the preceding $N-1$ words, such as approximating $P(\text{wonder}|\text{Unicorns bring joy and})$ as $P(\text{wonder}|\text{and})$. For bigrams ($N=2$) and trigrams ($N=3$), the probability of a full sequence is calculated as the product of individual conditional probabilities. This approach simplifies computation by ignoring long-range dependencies, with extreme cases considering no context at all.

Slide 14: N-grams

The unigram model assumes word independence, calculating the joint probability of a sequence as the product of individual word probabilities, expressed as $P(w_1, ..., w_n) = \prod_i P(w_i)$ and $P(w_i|w_{1:n}) = P(w_i)$. This approach generates sentences by selecting words based solely on their frequency without considering context. The slide illustrates this limitation with an automatically generated sentence containing random, unconnected words like "To him swallowed confess hear both."

Slide 15: N-grams

The bigram model approximates sentence probability by assuming each word depends only on the immediately preceding word, utilizing special dummy tokens for sequence start and end. Probability calculations involve counting specific bigram occurrences relative to total instances of the preceding word, where the vocabulary size includes these boundary tokens. To address data sparsity, smoothing techniques add one to the numerator and the vocabulary size to the denominator for all probability components.

Slide 16: N-grams

Maximum Likelihood Estimate (MLE) determines language model parameters by maximizing the likelihood of a training set given those parameters. Intuitively, the probability of a word like "bagel" appearing 400 times in a one-million-word corpus is estimated as 400/1,000,000, or 0.0004. While this estimate is specific to the training data and may not generalize perfectly, it represents the value that makes the observed training corpus most probable.

Slide 17: N-grams

This slide presents the Maximum Likelihood Estimation (MLE) for bigrams within language modeling. It approximates the probability of a word given the entire history as the probability given only the immediately preceding word, expressed as $P(w_n|w_{1:n}) \approx P(w_n|w_{n-1})$. The formula calculates this conditional probability by dividing the count of the bigram (w_{n-1}, w_n) by the total count of the preceding word w_{n-1}.

Slide 18: N-grams

Multiplying many small probabilities in large corpora or long sentences can cause numerical underflow. To address this, n-gram language models utilize log probabilities, converting the product of individual probabilities into a sum of their logarithms. If actual probability values are required, the exponential function is applied to the resulting sum of log probabilities.

Slide 19: N-grams

N-gram models face two primary limitations: they cannot capture long-distance dependencies without impractically high n-values that cause sparsity, and they estimate word probabilities independently, failing to recognize semantic similarities like those between "cat" and "dog." Large language models address these inefficiencies by handling extended context and representing similar words similarly to capture meaning.

Slide 20: Generation: sampling

Slide 20 presents the "Generation: sampling" section within the broader language modeling outline, which also includes introduction, n-grams, evaluation, smoothing, and summary. As no specific professor explanation or detailed content was provided for this slide in the source material, it serves solely as a structural heading indicating the upcoming topic of sampling methods for text generation.

Slide 21: Generation: sampling

Shannon (1948) proposed generating text by randomly selecting a starting unit and sequentially recording the succeeding unit found in a corpus, a method adapted in NLP to sample words based on their probabilities. Frequent words like "the" occupy larger segments of a probability distribution axis, making them more likely to be selected when a random point is chosen. While computers handle this probabilistic sampling efficiently, humans often simplify the process by selecting only the most probable word at each step.

Slide 22: Generation: sampling

Slide 22 presents a visualization of language model generation through sampling based on probabilities. This concept is attributed to Jurafsky and Martin (2025) within the context of language modeling. No additional professor explanation or specific technical details regarding the sampling mechanism are provided in the aligned source material for this section.

Slide 23: Generation: sampling

Text generation using a trained bigram model begins by selecting a random initial bigram (<s>, w) based on the probability p(w|<s>). The process repeats by choosing subsequent bigrams (w, x) according to p(x|w), continuing until a bigram ending with the end-of-sequence token </s> is selected. This sampling method prioritizes frequent words like "the" over infrequent ones like "polyphonic," as illustrated by the example sequence starting with "unicorns."

Slide 24: Generation: sampling

Text generation via sampling begins by selecting a random initial word based on the probability $p(w|\langle s\rangle)$. Subsequent words are chosen sequentially according to their conditional probabilities, such as $p(x|w)$, extending the sequence step-by-step. This iterative process continues until a bigram ending with the end-of-sentence token $\langle /s\rangle$ is selected, completing the generated sentence.

Slide 25: Generation: sampling

While standard sampling draws from the full probability distribution, alternative methods avoid generating words from the unlikely tail to prevent improbable outputs like "polyphonic." Key techniques include Temperature sampling (Ficler and Goldberg, 2017), Top-k sampling (Fan et al., 2018), and Top-p sampling (Holtzman et al., 2020). These approaches refine text generation by restricting the selection pool beyond simple probability-based sampling.

Slide 26: Generation: sampling

Slide 26 illustrates sentence generation using N-gram models trained on Shakespeare text, demonstrating a progression from unigrams producing probable words to bigrams forming coherent sequences and trigrams memorizing actual sentences. The professor notes that while higher-order n-grams improve modeling accuracy, they increasingly rely on analyzing and reproducing the training data. This section also briefly mentions sampling methods like temperature and top-k as alternatives to standard generation, with a detailed trade-off between deterministic and creative sampling reserved for future discussion.

Slide 27: Generation: sampling

Slide 27 presents examples of sentences generated through sampling using N-gram models trained on the Wall Street Journal (WSJ) corpus. This content is sourced from Jurafsky and Martin (2025) within the context of language modeling in natural language processing. The slide serves to illustrate the output capabilities of these statistical generation methods without providing specific sample sentences or further technical details in the provided source material.

Slide 28: Evaluation

Slide 28 presents the evaluation outline for the language modeling lecture, listing key topics including introduction, n-grams, generation via sampling, evaluation, smoothing, and a summary. This slide serves as a structural overview of the course content without providing specific definitions or detailed explanations for these concepts.

Slide 29: Evaluation

Extrinsic or downstream evaluation compares language models by integrating them into specific tasks, such as machine translation or speech recognition, to measure and compare their task accuracies. While this method identifies the superior model for a given application, it is limited by being expensive and time-consuming, with results that may not generalize to other applications.

Slide 30: Evaluation

Intrinsic evaluation uses perplexity to directly measure a language model's performance at predicting words, the task it is trained on. While lower perplexity indicates better prediction accuracy and provides a practical, single general metric for models including LLMs, it does not necessarily correspond to real application or downstream task performance. Despite this limitation, perplexity remains the most commonly used evaluation method due to its ease of use compared to training separate models for specific tasks.

Slide 31: Evaluation

Model parameters are trained on a training set, while performance is measured on an unseen test set using metrics like perplexity to assess generalization. Test sets must reflect specific task languages or ensure diversity for general-purpose models, strictly avoiding data contamination where test sentences appear in the training data. To prevent overfitting to benchmark characteristics, models should be tuned repeatedly on a development set, reserving the test set exclusively for final evaluation.

Slide 32: Evaluation

Repeatedly testing on the test set risks tuning the model specifically to its characteristics, so it must be reserved exclusively for final evaluation. Instead, a separate development set should be utilized for all intermediate evaluations during the training process. This separation ensures that the final performance assessment remains unbiased and accurately reflects the model's generalization capabilities.

Slide 33: Evaluation

Evaluation in language modeling relies on the intuition that a good model prefers "real" sentences by assigning them higher probabilities compared to "word salad" or rarely observed sequences. This concept is verified by trying different values to ensure the model correctly distinguishes between frequently observed and nonsensical text.

Slide 34: Evaluation

Evaluation relies on perplexity, based on the intuition that a good language model predicts upcoming words by assigning high probabilities to likely continuations. Even if the actual next word (e.g., "anchovies") is not the most probable prediction (e.g., "mushrooms"), the model receives credit proportional to the assigned probability compared to unlikely alternatives. This approach requires averaging performance across many instances rather than judging based on single difficult cases where the correct word has low probability.

Slide 35: Evaluation

The best language model assigns a high probability to the entire test set. To compare two models, compute and contrast their probabilities for the test set; the superior model yields the higher probability because it is less surprised by the data.

Slide 36: Evaluation

Raw probability is unsuitable for evaluating language models because the likelihood of a test set decreases as its length increases. Consequently, evaluation requires a per-word metric that normalizes scores by sequence length to ensure fair comparison across different text sizes. This approach addresses the inherent bias where longer sequences are naturally assigned lower probabilities.

Slide 37: Evaluation

Perplexity evaluates language models by measuring how surprised they are by an unseen test set, defined as the inverse probability of the sequence normalized by word count. Unlike raw probability, which decreases with sentence length, perplexity provides a per-word metric where lower values indicate better performance, equivalent to maximizing probability. While unigram models yield high perplexity due to lacking context, n-gram models like bigrams and trigrams achieve progressively lower scores with diminishing returns for higher N.

Slide 38: Evaluation

Slide 38 defines the evaluation of bi-gram language models using Perplexity, denoted as PP(W). The formula calculates perplexity as the N-th root of the inverse probability of a sequence W, which simplifies via the chain rule to the product of inverse conditional probabilities P(wi|wi−1) for bigrams. This metric quantifies how well the model predicts a test set by measuring the uncertainty of the predicted word sequence.

Slide 39: Evaluation

This slide presents an evaluation example of language modeling using perplexity scores on the WSJ corpus, which consists of 38 million training words and 1.5 million test words. The results demonstrate that as the n-gram order increases from unigram to trigram, perplexity decreases significantly from 962 to 109, with a further drop to 38 for higher-order models. This trend illustrates that incorporating more context through higher n-gram orders improves model performance by reducing uncertainty.

Slide 40: Smoothing

Slide 40 presents the outline for the smoothing topic within the language modeling module. The agenda covers an introduction, n-grams, generation via sampling, evaluation, smoothing techniques, and a summary. No specific definitions or detailed explanations are provided in the source material for this section.

Slide 41: Smoothing

Using Shakespeare as a corpus with 884K tokens and 29K types reveals extreme data sparsity in language modeling. Although there are 844 million possible bigrams, only 300K actually occur, meaning 99.96% of potential combinations have zero entries. This sparsity problem intensifies significantly for longer n-grams, such as 4-grams, necessitating smoothing techniques to handle unseen events.

Slide 42: Smoothing

Smoothing addresses generalization in language modeling by handling unseen word sequences. In this example, training data includes "ate" followed by "lunch," "dinner," "a," and "the," but not "breakfast." Consequently, without smoothing, the probability of "breakfast" given "ate" is calculated as zero for test sentences containing this unseen combination.

Slide 43: Smoothing

Smoothing addresses the issue where unseen bigrams in training data result in zero probabilities, making perplexity calculation impossible due to division by zero. For instance, if "ate breakfast" never appears in training, its probability is zero, preventing evaluation on test sets. Smoothing mechanisms redistribute probability mass from frequent events to rare or unseen ones, often using an additive constant k between zero and one rather than the standard add-one approach, which can distort counts too severely for n-gram models.

Slide 44: Smoothing

Slide 44 introduces smoothing as a technique to address sparse statistics in language modeling, illustrated by the conditional counts following "denied the" (e.g., 3 allegations, 2 reports). The slide adapts an example from Dan Klein and summarizes the core concept of smoothing with the phrase "Steal from the rich to give to the poor," indicating a redistribution of probability mass from frequent events to rare or unseen ones.

Slide 45: Smoothing

Smoothing in language modeling redistributes probability mass from high-frequency events to zero or low-probability events, a concept described as "stealing from the rich to give to the poor." This process adjusts raw counts, such as those shown for words following "denied," to ensure no event has a zero probability. The simplest implementation of this technique is known as add-one or Laplace smoothing.

Slide 46: Smoothing

Slide 46 introduces Add-one (Laplace) smoothing as a technique to address zero probabilities in language modeling. It contrasts the Maximum Likelihood Estimate (MLE), which calculates probability based solely on observed counts, with Laplace smoothing, which adds one to every count in the numerator and adjusts the denominator by adding the vocabulary size V. This adjustment ensures that unseen word sequences receive a non-zero probability.

Slide 47: Smoothing

Add-one smoothing is considered a blunt instrument because it drastically alters counts, making it unsuitable for N-gram language models. It is primarily used in NLP applications where the number of zero counts is not significant. While an alternative approach using add-k (where 0 < k < 1) exists, it also fails to perform well effectively.

Slide 48: Smoothing

Smoothing employs two primary methods: backoff and interpolation. Backoff uses trigrams when evidence is sufficient, otherwise falling back to bigrams or unigrams based on data sparsity. Interpolation mixes unigram, bigram, and trigram probabilities regardless of evidence availability. Empirically, interpolation has been found to work better than backoff.

Slide 49: Smoothing

Stupid backoff estimates n-gram probabilities by recursively falling back to lower-order models (e.g., from trigram to bigram to unigram) when higher-order counts are zero. However, this approach is probabilistically unsound because it causes the sum of probabilities to exceed one by adding mass without discounting higher-order terms. To correct this violation, techniques like Katz backoff are required to properly discount higher-order n-gram probabilities and ensure a valid probability distribution.

Slide 50: Smoothing

Smoothing via simple interpolation estimates the probability of a word by combining trigram, bigram, and unigram probabilities. The formula is $\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n|w_{n-2}w_{n-1}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n)$, where the weights λ_i sum to one. This approach integrates lower-order n-gram models to handle data sparsity in language modeling.

Slide 51: Smoothing

Smoothing via interpolation estimates the probability of a word given its two preceding words by combining trigram, bigram, and unigram probabilities. The formula uses context-conditioned lambda weights to linearly combine these n-gram models, allowing the system to back off to lower-order statistics when higher-order contexts are sparse or unseen.

Slide 52: Smoothing

Smoothing via interpolation combines trigram, bigram, and unigram probabilities using weights (λ) that sum to one for probabilistic correctness. While λs can be context-dependent, they are typically determined by fixing N-gram probabilities on training data and searching for values that maximize the probability of a held-out development set. This process optimizes the model's performance on unseen data without altering the underlying N-gram counts.

Slide 53: Smoothing

Smoothing does not resolve the issue of out-of-vocabulary (OOV) words unseen in training data. In practice, both unseen and low-frequency words are replaced by a dummy <UNK> token. Consequently, probabilities for these rare or unknown terms are estimated based on this single <UNK> token rather than individual word counts.

Slide 54: Summary

This slide summarizes the lecture on language modeling by outlining the key topics covered: introduction, n-grams, generation via sampling, evaluation, and smoothing. It serves as a concluding overview of the course material presented in this session.

Slide 55: Summary

This slide summarizes the key aspects of N-gram language models, covering estimation via Maximum Likelihood Estimation (MLE), generation through sampling, and evaluation using perplexity. It highlights the importance of selecting appropriate training and test data to ensure generalization. Finally, it notes the inherent limitations of MLE and introduces workarounds to address these issues.

Slide 56: Summary

Large Language Models (LLMs) utilize neural networks for estimation and generation, effectively capturing meaning and handling long-distance dependencies. These models are evaluated using modified perplexity, though generalization concerns persist. For further details on these topics, refer to Lecture 4.

Slide 57: Summary

Slide 57 provides a reference list for the lecture on language modeling, citing foundational and contemporary works in natural language processing. Key sources include Shannon's 1948 mathematical theory of communication, Jurafsky and Martin's 2025 textbook, and recent papers on hierarchical story generation, stylistic control, and neural text degeneration. These references support the course material without additional explanatory content from the transcript.

Key points to remember

To address data sparsity, the lecture focuses on smoothing techniques that redistribute probability mass from frequent events to rare or unseen ones, summarized as "stealing from the rich to give to the poor." While Add-One (Laplace) smoothing ensures non-zero probabilities by adding one to all counts, it is often considered a blunt instrument that distorts distributions too severely for n-gram models. Consequently, more sophisticated methods like Backoff and Interpolation are preferred; Backoff recursively falls back to lower-order n-grams when higher-order evidence is insufficient, while Interpolation linearly combines probabilities from unigram, bigram, and trigram models using context-conditioned weights. The lecture emphasizes that proper smoothing must ensure probabilistic soundness, such as through Katz backoff, to maintain a valid probability distribution where the sum of probabilities equals one.